

OFA-ii: Numbers as Iterators — Reference Spec v1

Tagline. Natural numbers are the *initial iterator*. Iteration, arithmetic, and Church encodings fall out as consequences of initiality—not as ad-hoc tricks.

0) Map of the spec

- **Foundations:** categorical definition (NNO as initial $1 + (-)$ -algebra), internal vs external naturals.
 - **Core APIs:** Haskell and TypeScript baselines built from `foldN`.
 - **Arithmetic:** definitions via folds; Church numerals; iterator-of-iterator law.
 - **Interpretations:** backends (strings, lists/state, cyclic groups) + faithfulness tests.
 - **Laws & Tests:** property suites (QuickCheck / fast-check) and equational schemas.
 - **Fusion/Optimization:** rewrite rules, iterator fusion, exponentiation-by-squaring (external level), parametricity notes.
 - **Extensibility:** categorical interface, alternative encodings, future directions.
-

1) Foundations (clear levels)

1.1 Natural Numbers Object (NNO)

An NNO is an object $(\mathbb{N}, z, s)$ with $z : 1 \rightarrow \mathbb{N}$, $s : \mathbb{N} \rightarrow \mathbb{N}$ such that for any $q : 1 \rightarrow A$, $f : A \rightarrow A$ there exists a **unique** $u : \mathbb{N} \rightarrow A$ with

$$u \circ z = q, \quad u \circ s = f \circ u.$$

This is the *recursion principle*; uniqueness entails induction.

1.2 Iteration derives from initiality

Inductively, $u \circ s^n = f^n \circ u$. With $u = z$, one reads $s^n \circ z = (\text{run } n \text{ times starting at } q)$. Thus the iterator view $n \mapsto f^n$ **emerges** from initiality.

1.3 Levels

- **Internal numerals:** morphisms $\bar{n} : 1 \rightarrow \mathbb{N}$ (e.g., $\bar{2} = s \circ s \circ z$).
 - **External naturals:** metalanguage datatype (Peano). They *drive* folds in code.
 - **Interpretation:** choosing a carrier N and step $s : N \rightarrow N$ yields a representation $n \mapsto s^n \in \text{End}(N)$.
-

2) Core APIs from the recursion principle

2.1 Haskell baseline

```
{-# LANGUAGE RankNTypes #-}
module OFA.Core where

-- External naturals
data Nat = Z | S Nat deriving (Eq, Show)

-- Recursion principle (fold)
foldN :: a -> (a -> a) -> Nat -> a
foldN z _ Z      = z
foldN z f (S n) = f (foldN z f n)

-- Church encoding emerges
type ChurchNat = forall a. (a -> a) -> (a -> a)

asChurch :: Nat -> ChurchNat
asChurch n = \f -> foldN id (f .) n -- f^n

-- Arithmetic via folds
plus :: Nat -> Nat -> Nat
plus m n = foldN n S m           -- m + n = S applied m times to n

times :: Nat -> Nat -> Nat
\t -> foldN Z (plus t) -- n |-> n + t, repeated m times

-- Conversions (for tests & IO)
fromNat :: Nat -> Integer
fromNat Z      = 0
fromNat (S n) = 1 + fromNat n

toNat :: Integer -> Nat
toNat k | k <= 0 = Z
        | otherwise = S (toNat (k-1))

-- Iterators over a chosen carrier
asIterator :: (x -> x) -> Nat -> (x -> x)
asIterator step n = foldN id (step .) n

-- Interpretation with a basepoint
value :: (x -> x) -> x -> Nat -> x
value step base n = foldN base step n
```

Note. `times` uses a higher-rank idiom: fix the *addend* first; then fold over the multiplier. This aligns with the iterator-of-iterator law below.

2.2 TypeScript baseline

```
// External naturals
export type Nat = { _tag: 'Z' } | { _tag: 'S', pred: Nat };
export const Z: Nat = { _tag: 'Z' };
export const S = (n: Nat): Nat => ({ _tag: 'S', pred: n });

// Recursion principle
export const foldN = <A>(z: A, f: (a: A) => A, n: Nat): A =>
  n._tag === 'Z' ? z : f(foldN(z, f, n.pred));

// Church encoding
export type ChurchNat = <A>(f: (a: A) => A) => (a: A) => A;
export const toChurch = (n: Nat): ChurchNat => f => x => foldN(x, f, n);

// Arithmetic
export const plus = (m: Nat, n: Nat): Nat => foldN(n, k => S(k), m);
export const times = (m: Nat, n: Nat): Nat => foldN<Nat>(Z, k => plus(k, n), m);

// Iterator & evaluation
export const asIterator = <X>(step: (x: X) => X, n: Nat) => (x: X) => foldN(x,
  step, n);
export const value = <X>(step: (x: X) => X, base: X, n: Nat): X => foldN(base,
  step, n);

// Helpers
export const toNat = (k: number): Nat => (k <= 0 ? Z : S(toNat(k - 1)));
export const fromNat = (n: Nat): number => foldN(0, x => x + 1, n);
```

3) The iterator laws (clean and typed)

3.1 Composition (addition)

For any endomap `f`,

```
asIterator f (plus m n) = asIterator f m . asIterator f n
```

Equivalently for Church numerals: $(m \oplus n) f = (m f) \circ (n f)$.

3.2 Iteration-of-iteration (multiplication)

```
asIterator f (times m n) = asIterator (asIterator f n) m
```

I.e., $(f^n)^m = f^{mn}$ with equality in the endomap monoid.

3.3 Identity laws

```
plus Z n = n,      times Z n = Z,      times (S Z) n = n.
```

4) Interpretations & faithfulness

4.1 Interpreter record (Haskell)

```
module OFA.Interpret where

data Interpreter a = Interp
  { step    :: a -> a  -- s : N -> N
  , base    :: a      -- z : 1 -> N
  , eqTest  :: a -> a -> Bool
  }

interpret :: Interpreter a -> Nat -> a
interpret (Interp s b _) n = foldN b s n

isFaithfulUpto :: Interpreter a -> Int -> Bool
isFaithfulUpto it k =
  let xs = [ interpret it (toNat i) | i <- [0..k] ]
  in pairwiseDistinct (eqTest it) xs

pairwiseDistinct :: (a -> a -> Bool) -> [a] -> Bool
pairwiseDistinct eq = go
  where go []      = True
        go (y:ys) = all (not . eq y) ys && go ys
```

4.2 Example backends

```
-- Strings: building chains of "*"
stringInterp :: Interpreter String
stringInterp = Interp { step = (++ "*"), base = "", eqTest = (==) }
```

```

-- Natural numbers as counts (Set)
intInterp :: Interpreter Int
intInterp = Interp { step = (+1), base = 0, eqTest = (==) }

-- Cyclic group: modulo 5 (non-faithful by design)
mod5Interp :: Interpreter Int
mod5Interp = Interp { step = (`mod` 5) . (+1), base = 0, eqTest = (==) }

```

Reading: Faithfulness checks the *representation* $n \mapsto s^n$ into `End(N)`. It is optional and not part of the NNO definition.

5) Laws & property tests

5.1 Haskell (QuickCheck)

```

module OFA.Laws where
import Test.QuickCheck
import OFA.Core

-- Arbitrary Nat
instance Arbitrary Nat where
  arbitrary = toNat . abs <$> arbitrary

-- 1) Recursion uniqueness (extensional): value ≡ foldN
prop_value_fold :: (Int -> Int) -> Int -> Nat -> Bool
prop_value_fold f q n = value f q n == foldN q f n

-- 2) Iterator composition (addition)
prop_iter_plus :: (Int -> Int) -> Nat -> Nat -> Int -> Bool
prop_iter_plus f m n x =
  asIterator f (plus m n) x == (asIterator f m . asIterator f n) x

-- 3) Iteration-of-iteration (multiplication)
prop_iter_times :: (Int -> Int) -> Nat -> Nat -> Int -> Bool
prop_iter_times f m n x =
  asIterator f (times m n) x == asIterator (asIterator f n) m x

-- 4) Church arithmetic agrees
prop_church_plus :: Nat -> Nat -> (Int -> Int) -> Int -> Bool
prop_church_plus m n f x =
  toChurch (plus m n) f x == (toChurch m f) ((toChurch n f) x)

```

5.2 TypeScript (fast-check)

```
import * as fc from 'fast-check';
import { Z, S, foldN, plus, times, asIterator, value, toNat, fromNat } from './core';

const natArb: fc.Arbitrary<number> = fc.nat(200); // keep small for stack safety
const toPeano = (k: number) => toNat(k);

// value ≡ foldN
fc.assert(fc.property(natArb, natArb, natArb, (fSeed, q, n) => {
  const f = (x: number) => x + (fSeed % 3); // simple affine step to avoid
function eq
  const peano = toPeano(n);
  return value(f, q, peano) === foldN(q, f, peano);
})));

// iterator composition (addition)
fc.assert(fc.property(natArb, natArb, natArb, (fSeed, m, n) => {
  const f = (x: number) => x + (1 + (fSeed % 5));
  const M = toPeano(m), N = toPeano(n);
  const x0 = 0;
  return asIterator(f, plus(M, N))(x0) === (asIterator(f, M)(asIterator(f, N)
(x0)));
})));
```

Note on function equality: we test via *sampling points* (e.g., integers) rather than extensional equality of functions, which is undecidable.

6) Fusion & optimization

6.1 Iterator fusion

Rewrite rule (safe in any eager language):

```
asIterator f m ◦ asIterator f n ⇒ asIterator f (plus m n)
```

This deforests intermediate states when chaining loops.

6.2 Exponentiation by squaring (external, optional)

A performance variant for `asIterator` can use a binary external counter (e.g., `Integer`) to obtain $O(\log n)$ composition count. This is an *implementation optimization* and does not change the equational theory.

6.3 Parametricity free theorems

`ChurchNat =  $\forall A. (A \rightarrow A) \rightarrow (A \rightarrow A)$` ensures

$(m == n) \text{ iff } \forall A, f. m \ f \equiv n \ f$

so Church equality is observational, enabling lightweight proofs by polymorphism.

7) Categorical interface (optional layer)

```
class HasNNO cat where
  type N cat :: *
  zero :: cat () (N cat)
  succ :: cat (N cat) (N cat)
  fold :: cat () a -> cat a a -> cat (N cat) a -- the universal arrow
```

This interface abstracts the `foldN` principle to general categories (e.g., topos-like settings, effectful categories, etc.).

8) Worked examples

1. **Strings:** `step = (++ "*")`, `base = ""`. Faithful; `fromNat n` matches string length.
2. **State counters:** `step = (+1)`, `base = 0`. Faithful; recovers usual arithmetic by evaluation.
3. **Cyclic rotation:** `step = (x -> x+1 mod 5)`. Non-faithful; illustrates collapse of representation while laws still hold.

9) What ships in v1.0

- `ofa-core` (Haskell): `Nat`, `foldN`, `asChurch`, `plus`, `times`, `asIterator`, `value`.
- `ofa-interpret` (Haskell): `Interpreter`, `interpret`, `isFaithfulUpto`, example backends.
- `ofa-laws` (Haskell): QuickCheck suite with the laws above.
- `ofa-ts` (TypeScript/ESM): mirrors of `core` + `fast-check` tests.
- Docs with level discipline + law statements.

10) Future directions

- **General recursion schemas:** parametrized folds (catamorphisms), and course-of-value iteration.
 - **Indexed/graded naturals:** track resource measures in the type of numerals.
 - **Effectful categories:** interpret in Kleisli categories (e.g., iteration with effects) while preserving laws.
 - **Proof artifacts:** Agda/Coq mechanization of initiality → iterator laws → arithmetic.
 - **Optimizer:** rule-based fusion and iterator-of-iterator collapsing in a compiler plugin.
-

End of Spec v1.

11) References & Further Reading (External)

Core category theory & NNOs

- Mac Lane, S. *Categories for the Working Mathematician*, 2nd ed., Springer (1998). Classic reference; NNOs and initial algebras. <https://www.springer.com/gp/book/9780387984032>
- Awodey, S. *Category Theory*, 2nd ed., Oxford University Press (2010). Accessible treatment of NNOs and recursion. (Preview PDF available.) <https://awodey.github.io/typetheory/notes/typetheory3b.pdf>
- nLab: **Natural numbers object**. Concise encyclopedia entry with diagrams and variants beyond **Set**. <https://ncatlab.org/nlab/show/natural%2Bnumbers%2Bobject>

Church numerals & encodings

- Barendregt, H. P. *The Lambda Calculus: Its Syntax and Semantics*, North-Holland (1984/1986). Canonical background on Church encoding.
- Wikipedia: **Church encoding**. Pragmatic overview with formulas and examples. https://en.wikipedia.org/wiki/Church_encoding
- Pierce, B. C. *Types and Programming Languages*, MIT Press (2002). Church encodings and folds in typed settings. (Contents/preview) <https://www.cis.upenn.edu/~bcpierce/tapl/contents.pdf>

Parametricity / “Theorems for Free”

- Wadler, P. **Theorems for Free!** (1989). The seminal paper deriving free theorems from polymorphic types. <https://people.mpi-sws.org/~dreyer/tor/papers/wadler.pdf>
- Ahmed, A. et al. **Theorems for Free for Free** (2017). Modern parametricity with cast calculi. <https://www.ccs.neu.edu/home/amal/papers/thmfreefree.pdf>

Recursion schemes & catamorphisms

- Meijer, Fokkinga, Paterson. **Functional Programming with Bananas, Lenses, Envelopes and Barbed Wire** (1991). Catamorphisms and algebraic program calculation. <https://maartenfokkinga.github.io/utwente/mmf91m.pdf>
- Lambek, J.; Scott, P. J. *Introduction to Higher-Order Categorical Logic*, CUP (1986/1988). NNOs as initial F-algebras; categorical logic backdrop. <https://www.cambridge.org/us/universitypress/subjects/>

[mathematics/logic-categories-and-sets/introduction-higher-order-categorical-logic?format=PB&isbn=9780521356534](https://www.cambridge.org/9780521356534/mathematics/logic-categories-and-sets/introduction-higher-order-categorical-logic?format=PB&isbn=9780521356534)

Property-based testing (for our law checks)

- Claessen, K.; Hughes, J. **QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs** (ICFP 2000). <https://www.cs.tufts.edu/~nr/cs257/archive/john-hughes/quick.pdf>
- **fast-check** (TypeScript): Official docs and integration guides. <https://fast-check.dev/>

These references anchor: (i) the initial-algebra view of naturals, (ii) Church encodings as iterators, and (iii) parametricity arguments that justify the iterator laws in the polymorphic setting.

Citizen Gardens © 2025 CC-NC-ND 4.0